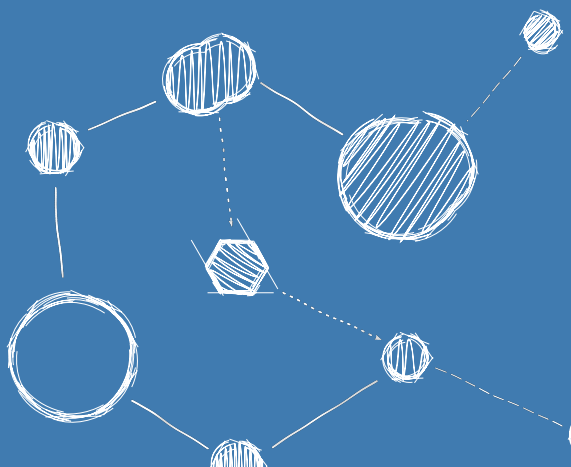
institut für astrophysik
UNIVERSITÄTSSTERNWARTE WIEN

universität wien

FLIGHTOS – AN RTOS FOR SPACE-BASED ASTRONOMY

Test Plan



Test Plan

Reference: FLIGHTOS-UVIE-TP-001

Version: Issue 1.1, August 31, 2026

Prepared by: Armin Luntzer[1]

Checked by: Roland Ottensamer[1]

Approved by: Franz Kerschbaum[1]

Authorised by: -

[1] Department of Astrophysics, University of Vienna

Contents

List of Requirements

Revision History

Revision	Date	Author(s)	Description
0.1	23.07.2017	AL	initial version
0.2	01.07.2016	AL	incorporate changes from review items
1.0	01.06.2017	FK	document approved
1.1	31.08.2026	AL	adopt FlightOS name

1. Introduction

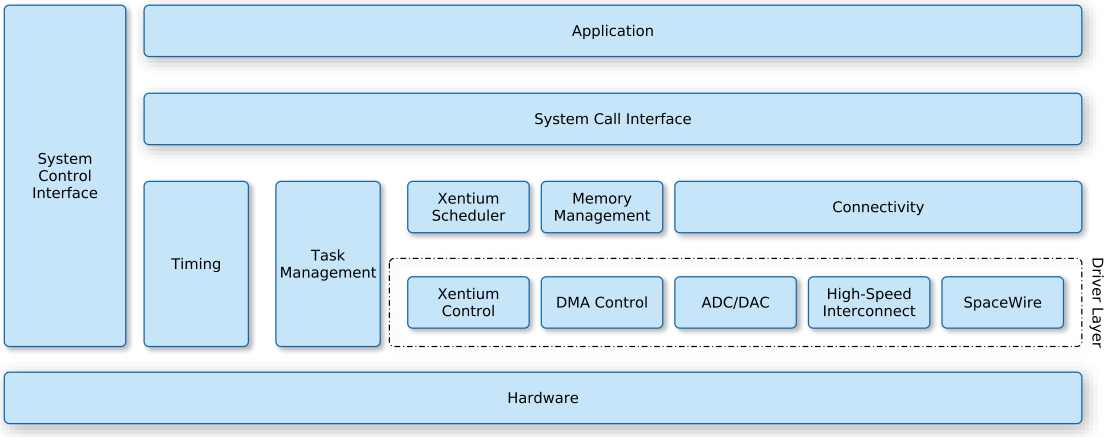


Figure 1.1: The logical model of FlightOS. Here, the "hardware" layer represents both the hardware and the hardware abstraction layer of the software.

FlightOS[3][1][2] is an operating system targeting LEON3-based platforms such as *GR712RC* and *GR740*, but also has legacy support for drivers and data processing on the Xentium Digital Signal Processors (DSPs) of the Scalable Sensor Data Processor (SSDP).

In FlightOS, hardware is accessed in multiple layers of abstraction (see Figure 1.1). Typical Central Processing Unit (CPU) tasks such as thread/task management and timer operation are used as part of the operating system kernel and are also accessible by user applications via a system call interface. Other functional hardware components of the SSDP such as the Network On Chip (NoC) Direct Memory Access (DMA) have their own driver modules. These are in turn used by the Xentium scheduler and other higher level modules in the operating system. Configuration of and access to the latter from user space is done via a system call interface. The system control interface serves as an intermediate between all layers of the operating system, where system or module states and hardware modes or usage statistics may be exported by individual components for external (user) access.

The compliance of FlightOS with its requirements and its operational environment are partially verified by testing.

1.1 Purpose of the Document

This document defines test procedures for the FlightOS operating system. In particular, it defines the:

- types of tests
- environment in which a test is executed
- coverage of the test
- entity responsible for performing the test

1.2 Incremental Testing

FlightOS is not envisaged to be implemented in one final form. Instead, it is marked with version numbers for each release. This means that testing is version-dependent and is be applied in increments by executing a sub-set of tests only as modifications are made.

1.3 Test Sequences

Tests defined in this document are executed by automated test sequences in the form of software programs that control the execution of a test, check the correctness of the results and generate a summary report of the test case.

1.4 Items Under Test

Components and software modules of the operating system are tested on an unit-level where applicable. System level tests are tailored to the various configuration options available. and shall be accompanied by configuration files so the proper build may generated.

R-IUT-0001	Short Text	Software Requirement
	Hardware constraints	The test plan specified by the present requirements shall apply to all parts of FlightOS.

1.5 Operational Constraints

Each test shall specify the minimum hardware platform on which it must be performed. Unit-level test of platform-independent components may be executed on any compatible hardware. Platform-dependent tests may require specific hardware models (e.g. the SSDP for testing of the

NoC) or just any compatible processor model (e.g. LEON3 for testing the operation of the Memory Management Unit (MMU)).

R-OPC-0001	Short Text	Software Requirement
	Hardware constraints	The execution of the tests defined in this document shall comply with all the operational constraints applicable to the hardware used in the tests.

2. Applicable and Reference Documents

[1] *FlightOS Architectural Design Document*. 2026.

[2] *FlightOS Software Requirements Specification*. 2026.

[3] Armin Luntzer et al. *A Lightweight Operating System for the SSDP*. 2016.
URL: https://indico.esa.int/event/102/timetable/?view=standard.

3. Terms, Definitions and Abbreviated Items

3.1 Acronyms

ADC	Analog to Digital Converter
API	Application Programming Interface
BSP	Board Support Package
CPU	Central Processing Unit
DAC	Digital to Analog Converter
DMA	Direct Memory Access
DSP	Digital Signal Processor
ELF	Executable and Linkable Format
FIFO	First In - First Out
FPU	Floating Point Unit
GCC	GNU Compiler Collection
ILP	Instruction Level Parallelism
ISR	Interrupt Service Routine
MMU	Memory Management Unit
MPPB	Massively Parallel Processor Breadboarding system
NGAPP	Next Generation Astronomy Processing Platform
NoC	Network On Chip
POSIX	Portable Operating System Interface
PUS	Packet Utilisation Standard
RAM	Random-Access Memory
RISC	Reduced Instruction Set Computing
RMAP	Remote Memory Access Protocol
RR	Round Robin
SMP	Symmetric Multiprocessing
SoC	System On Chip
SSDP	Scalable Sensor Data Processor
TCM	Tightly-Coupled Memory
VLIW	Very Long Instruction Word

3.2 Glossary

Analog to Digital Converter (Analog to Digital Converter (ADC))

An Analog to Digital Converter is a system that converts an analog signal into a quantized digital signal. Its counterpart is the Digital to Analog Converter (DAC).

Application Programming Interface (Application Programming Interface (API))

The Application Programming Interface defines how a developer can write a program that requests services from an operating system or application. APIs are implemented by function calls composed of verbs and nouns, i.e. a function to execute on an object.

Board Support Package (Board Support Package (BSP))

A Board Support Package is the implementation of a specific interface defined by the abstract layer of an operating system that enables the latter to run on the particular hardware platform.

Central Processing Unit (CPU)

The Central Processing Unit is the electronic circuitry that interprets instructions of a computer program and performs control logic, arithmetic, and input/output operations specified by the instructions. It maintains high-level control of peripheral components, such as memory and other devices.

Digital Signal Processor (DSP)

A Digital Signal Processor is a specialised processor with its architecture targeting the operational needs of digital signal processing.

Digital to Analog Converter (DAC)

A Digital to Analog Converter is a system that converts a quantized digital signal into an analog signal. Its counterpart is the ADC.

Direct Memory Access (DMA)

Direct Memory Access is a feature of a computer system that allows hardware subsystems to access main system Random-Access Memory (Random-Access Memory (RAM)) directly, thereby bypassing the Central Processing Unit (CPU).

Executable and Linkable Format (Executable and Linkable Format (ELF))

The Executable and Linkable Format is a common standard file format for executables, object code, shared libraries, and core dumps.

First In - First Out (First In - First Out (FIFO))

In FIFO processing, the ”head” element of a queue is processed first. Once complete, the element is removed and the next element in line becomes the new queue head.

Floating Point Unit (Floating Point Unit (FPU))

A co-processor unit that specialises in floating-point calculations.

GNU Compiler Collection (GNU Compiler Collection (GCC))

The GNU Compiler Collection is a compiler system produced by the GNU project. It is part of the GNU toolchain collection of programming tools.

Instruction Level Parallelism (Instruction Level Parallelism (ILP))

Instruction-level parallelism (ILP) is a measure of how many instructions in a computer program can be executed simultaneously by the CPU.

Interrupt Service Routine (Interrupt Service Routine (ISR))

An Interrupt Service Routine is a function that handles the actions needed to service an interrupt.

LEON2

The LEON2 is a synthesisable VHDL model of a 32-bit processor compliant with the SPARC V8 architecture. It is highly configurable and particularly suitable for System On Chip (SoC) designs. Its source code is available under the GNU LGPL license

LEON3

The LEON3 is an updated version of the LEON2, changes include Symmetric Multiprocessing (Symmetric Multiprocessing (SMP)) support and a deeper instruction pipeline

LEON3-FT

The LEON3-FT is a fault-tolerant version of the LEON3. Changes to the base version include autonomous error handling, cache locking and different cache replacement strategies.

Massively Parallel Processor Breadboarding system (Massively Parallel Processor Breadboarding system (MI

The Massively Parallel Processor Breadboarding system is a proof-of-concept design for a space-hardened, fault-tolerant multi-DSP system with various subsystems to build a powerful digital signal processing system with a high data throughput. Its distinguishing features are the Network On Chip (NoC) and the Xentium DSPs controlled by a LEON2 processor. It was developed under ESA contract 21986 by Recore Systems B.V.

Memory Management Unit (MMU)

A Memory Management Unit performs address space translation between physical and virtual memory pages and protects unprivileged access to certain memory regions.

Network On Chip (NoC)

A Network On Chip is a communication system on an integrated circuit that applies (packet based) networking to on-chip communication. It offers improvements over more conventional bus interconnects and is more scalable and power efficient in complex System On Chip (SoC) desgins.

Next Generation Astronomy Processing Platform (Next Generation Astronomy Processing Platform (NGAPP)

Next Generation Astronomy Processing Platform was an evaluation of the MPPB performed in a joint effort of RUAG Space Austria and the Department of Astrophysics of the University of Vienna. The project was funded under ESA contract 40000107815/13/NL/EL/f.

Packet Utilisation Standard (Packet Utilisation Standard (PUS))

The Packet Utilisation Standard addresses the end-to-end transport of telemetry and telecommand data between user applications on the ground and applications onboard a satellite. See also ECSS-E-70-41A.

Portable Operating System Interface (Portable Operating System Interface (POSIX))

The Portable Operating System Interface is a family of standards specified by the IEEE Computer Society for maintaining compatibility between operating systems.

Random-Access Memory (RAM)

Random-Access Memory is a type of memory where each memory cell may be accessed directly via their memory addresses.

Reduced Instruction Set Computing (Reduced Instruction Set Computing (RISC))

RISC is a CPU design strategy that intends to improve performance by combining a simplified instruction set with a microprocessor architecture that is capable of executing an instruction in a smaller number of clock cycles.

Remote Memory Access Protocol (Remote Memory Access Protocol (RMAP))

The Remote Memory Access Protocol is a form of SpaceWire communication that transparently communicates writes to memory mapped regions between different hardware devices.

Round Robin (Round Robin (RR))

Round Robin is a scheduling algorithm where time slices are assigned in equal poritions and in circular order. In the context of threads, priorities are usually only used to control re-scheduling order when a mutex is accessed by a thread.

Scalable Sensor Data Processor (SSDP)

The Scalable Sensor Data Processor (SSDP) is a next generation on-board data processing mixed-signal ASIC, envisaged to be used in future scientific payloads requiring high-performance on-board processing capabilities. It is built opon a heterogeneous multicore architecture, combining two Xentium DSP cores with a general-purpose LEON3-FT control processor in a Network On Chip (NoC).

SpaceWire

SpaceWire is a spacecraft communication network based in part on the IEEE 1355 standard of communications.

Symmetric Multiprocessing (SMP)

Symmetric Multiprocessing denotes computer architectures, where two or more identical processors are connected to the same periphery and are controlled by the same operating system instance.

System On Chip (SoC)

A System On Chip is an integrated circuit that combines all components of a computer or other electronic system into a single chip.

Tightly-Coupled Memory (Tightly-Coupled Memory (TCM))

Tightly-Coupled Memory is the local data memory that is directly accessible by a Xentium's load/store unit. It can be viewed as a completely program-controlled data cache.

Very Long Instruction Word (Very Long Instruction Word (VLIW))

Very Long Instruction Word is a processor architecture design concept that exploits Instruction Level Parallelism (ILP). This approach allows higher performance at a smaller silicone footprint compared to serialised instruction processors, as no instruction re-ordering logic to exploit superscalar capabilities of the processor must be integrated on the chip, but requires either code to be tuned manually or a very sophisticated compiler to exploit the full potential of the processor.

Xentium

The Xentium is a high performance Very Long Instruction Word (VLIW) DSP core. It operates 10 parallel execution slots supporting 32/40 bit scalar and two 16-bit element vector operations.

4. Types of Tests

In order to satisfy the verification and validation needs of the test campaign, the following types of tests are defined for FlightOS:

- unit and integration tests
- hardware-software tests

4.1 Unit and Integration Tests

FlightOS is developed as a collection of individual, interacting components. A unit test verifies the behaviour of a single component, while an integration test verifies the behaviour of a number of interacting components.

Unit and integration tests are white-box type tests that executes a component with a series of input conditions and verify the output or action for the expected response. These type of tests are typically independent from the hardware platform.

R-UIT-0001	Short Text	Software Requirement
	Unit/Integ. Tests	Unit and Integration Tests shall be defined and executed for FlightOS components.
Comment	Tests can make use of the modified *kselftest* framework provided in the FlightOS source tree.	

R-UIT-0002	Short Text	Software Requirement
	Test Suite	A unit and integration test suite shall be provided as a set of applications.

R-UIT-0003	Short Text	Software Requirement
	Automatic Test Execution	A single mechanism shall be provided that automatically runs all unit and integration tests in sequence.
Comment	The *kselftest* framework in the FlightOS source tree provides a *Makefile* setup that can be used for this purpose.	

R-UIT-0004	Short Text	Software Requirement
	Test Results	The test suite shall generate feedback as a human-readable report describing the outcome of each unit and integration test.

R-UIT-0005	Short Text	Software Requirement
	Automatic Testing	A feedback mechanism shall be provided that allows an external automated testing mechanism to execute and detect the results of unit and integration tests.
Comment	This is very useful for automated bisection of code versions, e.g. by using the *git bisect run …* functionality.	

4.2 Hardware-Software Tests

These tests differ from unit and integration tests in that a particular hardware platform is required for testing. All system-level integration tests fall into this category.

R-HST-0001	Short Text	Software Requirement
	Hardware Tests	Hardware-software tests shall be defined and executed for FlightOS components with the objective of verifying the behaviour of components which directly interact with the relevant target hardware.

R-HST-0002	Short Text	Software Requirement
	Test Suite	A hardware-software test suite shall be provided as a set of applications.

R-HST-0003	Short Text	Software Requirement
	Test Results	The hardware-software tests shall generate feedback as a human-readable report describing the outcome of test.

R-HST-0004	Short Text	Software Requirement
	Automatic Testing	A feedback mechanism shall be provided that allows an external automated testing mechanism to execute and detect the results of hardware-software tests.
Comment	This is very useful for automated bisection of code versions, e.g. by using the *git bisect run …* functionality.	

4.3 Acceptance Tests

Acceptance tests are performed by the user on the integrated processing unit. Their objective is to exercise the selected FlightOS configuration while running in its operational environment. The operational environment for FlightOS is defined by the user according to their use case.

R-ACT-0001	Short Text	Software Requirement
	Acceptance Tests	Acceptance tests and their procedures shall be defined and executed for FlightOS.

R-ACT-0002	Short Text	Software Requirement
	Use Cases	The configuration and operational environment for an acceptance test of FlightOS shall be defined by the user according to their use case.

5. Test Environments

Test beds in which FlightOS tests are performed:

- general-purpose platform for regular unit and integration tests
- LEON2 and/or LEON3 platforms for hardware-software tests
- SSDP and/or MPPB for system-level hardware-software tests

The test environments required for each category of test are described in the following sections.

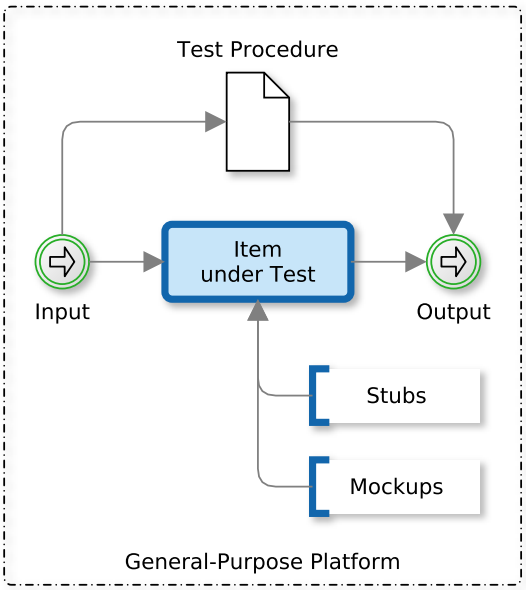


Figure 5.1: Unit and integration test environment. Stubs and Mockups are stand-ins for external dependencies with defined behaviour. A set of input data is fed into the item under test as specified in the test procedure.

5.1 Unit and Integration Test Environment

The test environment for unit and integration tests is shown in Figure 5.1.

The item under test is either one or a set of components of FlightOS. If needed, stub and mockup functions, which simulate the expected behaviour of external dependcies, are defined to create a representative environment in which the item under test is running.

Tests are controlled by a test sequence, which generates inputs for the item under test and collects and evaluates the generated output.

R-UITE-0001	Short Text	Software Requirement
	Unit Test Environment	Unit and integration tests shall be performed in an environment as defined in Figure 5.1.

5.2 Hardware-Software Test Environment

The hardware-software test environments are detailed on a per-test basis.

R-HSTE-0001	Short Text	Software Requirement
	HW/SW Test Environment	The required hardware-software test environment shall be defined in the specification of each individual test.

6. Test Coverage

This chapter defines the coverage of tests in the environments defined previously.

6.1 Source Code Coverage

Source code coverage must be achieved in unit and integration tests. A component is considered covered if full statement, decision and condition coverage is achieved.

If full coverage cannot be achieved, for example due to operational constraints, deviations are acceptable, provided a justification is given.

R-SCC-0001	Short Text	Software Requirement
	Statement Coverage	Unit and integration tests shall offer full statement coverage for FlightOS code.

R-SCC-0002	Short Text	Software Requirement
	Decision Coverage	Unit and integration tests shall offer full decision coverage for FlightOS code.

R-SCC-0003	Short Text	Software Requirement
	Condition Coverage	Unit and integration tests shall offer full condition coverage for FlightOS code.

R-SCC-0004	Short Text	Software Requirement
	Reduced Coverage	If the coverage specified in the previous requirements cannot be achieved due to operational contraints, justifications for deviations shall be provided for each of the individual conditions where the degree of coverage is below the specified goal.